

GENMED HUB™

System Architecture, Custom Algorithms & API Specifications Manual

A Comprehensive Guide to the Inner Workings of the Indian Generic Medicine Platform

Document Specification	Value / Context
Project Name	Generic Medicine Hub (GenMed Hub)
Core Architecture	Decoupled Client-Server (REST API)
Frontend Technology	React 19 + TypeScript + Vite + custom CSS
Backend Technology	Node.js + Express.js + ML Module
Local Mock Database	Flat JSON Database (db.json) with 520+ Medicines
Algorithms Implemented	Haversine Proximity, Simple Linear Regression with Seasonality, Anti-Counterfeit Verification Heuristics, Therapeutic Price Ratio Mapping, NLP Prescription Token Matcher
Target Environment	Windows Local Desktop / Development
Author	Antigravity AI (Pair Programming Assistant)
Date of Compilation	June 24, 2026

CONFIDENTIALITY NOTICE: The information contained in this technical document is proprietary. It is prepared for systems verification, developer reference, and educational inspection.

1. Executive Summary & System Architecture

GenMed Hub™ is an advanced digital healthcare platform engineered to bridge the awareness and accessibility gap in the Indian medicine market. The application is designed to solve a significant public healthcare issue: the massive price disparity between branded prescription medicines and chemically identical generic equivalents. The platform empowers users to search formulations, extract prescription listings via OCR, evaluate supply chain stock risks, authenticate batch numbers against counterfeit warning signs, locate governmental Jan Aushadhi (PMBJP) kendras, and forecast medicine demand trends.

The system relies on a decoupled, client-server web architecture to separate concerns, maximize scalability, and ensure high responsiveness:

A. Frontend Tier (React App Client):

A Single Page Application (SPA) compiled using the Vite bundler. It uses React 19 to provide responsive, state-driven interfaces and TypeScript to guarantee type safety across database query records, ML forecast graphs, and coordinates models. The client performs heavy operations locally—such as client-side OCR text extraction (using Tesseract.js), local storage purchase histories management (Health Wallet), and report generation (jsPDF)—to save bandwidth and maximize privacy.

B. Backend Tier (Express REST Server):

A Node.js server powered by Express.js. The server processes all incoming REST requests, handles route authentication, and implements the math modules for data analytics, demand forecasting, coordinate calculations, and batch risk assessments. It serves data from a highly realistic local JSON database simulating the Indian pharmaceutical landscape.

C. Local Mock Database System (db.json):

A simulated local database containing 520+ branded and generic medicines, 8 governmental and private pharmacies, and market trends records. The schema includes barcodes, side effects, contraindications, manufacturer shares, and seasonal sales volumes. The database operations are encapsulated in an ORM-like wrapper (db.js) utilizing standard JavaScript array algorithms (filtering, mapping, searching) and the Haversine formula for proximity calculations.

SYSTEM INTEGRATION ARCHITECTURE

The system operates with a RESTful communication model. The React client makes fetch requests to the Express server. The server runs specific computational models in Node.js (Simple Linear Regression and Heuristic risk rules) and reads/writes to a flat JSON file. This setup allows the entire stack to run locally without external cloud dependencies, enabling rapid offline utility in low-bandwidth Indian environments.

2. Technology Stack & Programming Languages

The platform utilizes modern web development technologies selected for rendering efficiency, development safety, and zero external deployment costs.

Subsystem	Technology / Library	Language Used	Functional Responsibility
Frontend Core	React 19.2.6	TSX / CSS / HTML	Modular UI components, state management, tab routing, dynamic user actions.
Frontend Build	Vite 8.0.12	TypeScript / JS	Fast compilation, hot module replacement, and production asset bundling.
OCR Processing	Tesseract.js 5.1.0	WebAssembly / JS	Client-side optical character recognition. Scans prescription images locally.
PDF Compilation	jsPDF 2.5.1	JavaScript	Compiles and downloads saving summary certificates dynamically in the browser.
Visual Charts	Recharts 2.12.7	TypeScript	Renders interactive graphs: sales forecasts, manufacturer shares, price ratios.
UI Icons	Lucide React 0.395.0	TypeScript	Sleek vector icons matching the dark/light design system.
Backend Core	Node.js v24.16.0	JavaScript	Asynchronous execution runtime hosting the API server and ML scripts.
REST Server	Express 4.19.2	JavaScript	Exposes endpoints, parses JSON bodies, routes requests, and manages CORS headers.
Data Storage	Flat JSON file	JSON Schema	Provides mock database tables, pre-seeded with 520+ medicines and stores.
AI Assistant	Gemini 2.5 Flash	JSON / HTTPS	Direct API integration for medicine consultations, fallback to offline rules.

3. System Data Model & Database Schema

The mock database (db.json) resides inside the server's data folder. It is dynamically generated if missing via ``server/generate_db.js``. The database consists of three main tables: Medicines, Stores, and Market Trends.

Table 1: Medicines

Contains comprehensive data on branded pharmaceuticals, their generic counterparts, and clinical parameters. Field descriptions are as follows:

Field Name	Type	Description / Schema Details
id	Number	Unique auto-incremented primary identifier key.
brandName	String	Commercial trade name (e.g. Crocin 650, Augmentin 625).
genericName	String	Chemical formulation name (e.g. Paracetamol 650mg, Amoxicillin + Clavulanic Acid).
composition	String	Detailed listing of active chemical molecules and quantities.
brandPrice	Number	Indian retail price in INR of the commercial branded version.
genericPrice	Number	Ceiling price in INR of the generic equivalent.
savings	Number	Pre-computed savings percentage: $((\text{brandPrice} - \text{genericPrice}) / \text{brandPrice}) * 100$
manufacturer	String	Pharmaceutical manufacturing company (e.g., GSK, Cipla, Alkem).
category	String	Therapeutic drug class (e.g. Cardiovascular, Antibiotics, Gastrointestinal).
dosage	String	Dosage form (e.g., Tablet, Capsule, Injection, Inhaler, Syrup).
availability	String	Current supply status: 'Available', 'Low Stock', or 'Out of Stock'.
schedule	String	Drug safety classification: 'OTC', 'Schedule H', or 'Schedule H1'.
sideEffects	Array (Str)	Adverse clinical side effects associated with the formulation.
contraindications	Array (Str)	Clinical conditions under which the drug must not be taken.
barcode	String	Simulated Indian barcode EAN-13 format starting with prefix '890'.

Table 2: Stores (Pharmacy Locations)

Stores are mapped with coordinates to support spatial queries. The structure contains: id (Number), name (String), type (String: 'Government Janaushadhi Store' or 'Private Generic Store'), rating (Number), phone (String), address (String), lat/lng (Numbers for coordinate system), and availableMedicines (Array of Medicine IDs in stock).

Table 3: Market Trends

Encapsulates datasets used to render financial charts and demand volumes: adoptionRates (year-by-year rate), categoryAnalysis (average prices, generic equivalents savings), regionalPricing (average price premium across North, South, West, East zones), monthlyVolume (historical demands in branded vs generic units), and manufacturersShare (pie chart percentage splits).

4. Custom Mathematical & Heuristic Algorithms

GenMed Hub incorporates several analytical models to support search, matching, spatial filtering, and forecasting. Below is a detailed breakdown of each algorithm.

4.1 Demand Forecasting via Simple Linear Regression & Seasonality

Mathematical Formulation: The system forecasts monthly branded and generic medicine demands over an upcoming horizon (usually 6 months). A Simple Linear Regression algorithm finds the line of best fit $y = mx + c$, where y represents the demand volume, and x represents the sequential month index. The slope m and intercept c are computed as follows:

$$\text{Slope (m)} = [N \cdot \sum(xy) - \sum x \cdot \sum y] / [N \cdot \sum(x^2) - (\sum x)^2]$$

$$\text{Intercept (c)} = [\sum y - m \cdot \sum x] / N$$

Seasonality Adjustment: Real-world healthcare demand experiences seasonal surges due to weather-induced illness spikes. The forecasting model applies multiplier factors corresponding to standard Indian seasons:

- **Monsoon Flu/Malaria Bump (July to September):** $+12\%$ factor (1.12 multiplier) representing spikes in antibiotic, respiratory, and fever medications.
- **Winter Cough/Cold Bump (December to January):** $+8\%$ factor (1.08 multiplier) representing respiratory and analgesic surges.

```
// Linear Regression computation in ml.js
function linearRegression(x, y) {
  const n = x.length;
  let sumX = 0, sumY = 0, sumXY = 0, sumXX = 0;
  for (let i = 0; i < n; i++) {
    sumX += x[i];
    sumY += y[i];
    sumXY += x[i] * y[i];
    sumXX += x[i] * x[i];
  }
  const slope = (n * sumXY - sumX * sumY) / (n * sumXX - sumX * sumX);
  const intercept = (sumY - slope * sumX) / n;
  return { slope, intercept };
}
```

4.2 Price Prediction Regression Model (Therapeutic Markups)

Algorithm Logic: The Price Predictor simulates generic equivalents pricing based on commercial branded prices. Branded markup margins vary heavily by therapeutic classes. The system applies a statistical mapping based on average generic price ratios in Indian markets (e.g. generic is $\sim 27\%$ of brand price in Antibiotics, $\sim 23\%$ in Cardiovascular, $\sim 30\%$ in Analgesics, etc.).

Boundary/Volume Discounts:

1. **Low-Cost Floor:** If brand price ≤ 30 \$ INR, generic price is set to 35% of brand price, subject to a minimum floor price of 3 \$ INR to represent base packaging costs.
2. **High-Cost Bulk Discount:** If brand price > 300 \$ INR, the generic markup rate is reduced by an additional 3% ($\text{Rate} - 0.03$) to account for bulk chemical drug manufacturing scale economies.

```
// Dynamic Price Prediction in ml.js
function predictGenericPrice(brandPrice, category = "General") {
  const markupRates = {
    "Antibiotics": 0.27, "Cardiovascular": 0.23, "Antidiabetics": 0.24,
    "Analgesics & Antipyretics": 0.30, "Antihistamines": 0.22, "General": 0.25
  };
  const rate = markupRates[category] || markupRates["General"];
  let predicted = brandPrice * rate;

  if (brandPrice < 30) {
    predicted = Math.max(3, brandPrice * 0.35);
  } else if (brandPrice > 300) {
    predicted = brandPrice * (rate - 0.03); // Bulk drug discount
  }
  return { predictedGenericPrice: parseFloat(predicted.toFixed(2)), savingsPercent: Math.round(((brandPrice - predicted) / brandPrice) * 100) };
}
```

4.3 Coordinate Proximity Calculation (Haversine Formula)

Mathematical Formulation: To calculate physical distances between a user's location and available pharmacies, the system cannot use simple Euclidean distance due to the earth's curvature. It implements the Haversine formula, which computes the great-circle distance between two pairs of latitudes and longitudes:

$$\begin{aligned} a &= \sin^2(\Delta\phi / 2) + \cos(\phi^1) \cdot \cos(\phi^2) \cdot \sin^2(\Delta\lambda / 2) \\ c &= 2 \cdot \arctan2(\sqrt{a}, \sqrt{1-a}) \\ d &= R \cdot c \end{aligned}$$

where $R = 6371 \text{ km}$ (Earth's radius), ϕ represents latitude in radians, and λ represents longitude in radians. Stores within the user-specified radius (default 20km) are filtered and sorted in ascending order of proximity.

```
// Distance calculation in db.js using Haversine formula
const toRad = (value) => (value * Math.PI) / 180;
const dLat = toRad(store.lat - lat);
const dLng = toRad(store.lng - lng);
const a =
  Math.sin(dLat / 2) * Math.sin(dLat / 2) +
  Math.cos(toRad(lat)) * Math.cos(toRad(store.lat)) *
  Math.sin(dLng / 2) * Math.sin(dLng / 2);
const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
const distance = 6371 * c; // returns distance in km
```

4.4 Prescription OCR Text Alignment & Keyword Matcher

NLP/Token Matching Logic: When a prescription is scanned via Tesseract.js, the text is sent to the backend. The matcher splits the raw text into line strings. For each line, it removes all punctuation/non-alphanumeric chars, converts words to lowercase, and splits them into discrete tokens. It then cross-references each token set with database medicines.

Strength Score: The keyword match strength is calculated as:

$$\text{Match Strength} = \left[\frac{\text{Count of overlapping tokens}}{\text{Total words in medicine brand name}} \right]$$

If the match strength is ≥ 0.5 (representing at least 50% keyword match) or if the line contains the exact brand name substring, the medicine is flagged as a match. This threshold accounts for minor OCR errors (e.g. 'Crocn' instead of 'Crocini').

```
// Token Matching snippet in server.js
for (const line of lines) {
  const tokens = line.toLowerCase().replace(/^[a-zA-Z0-9\s]/g, ' ').split(/\s+/);
  for (const medicine of medicines) {
    const brandWords = medicine.brandName.toLowerCase().split(/\s+/);
    const matchScore = brandWords.filter(word => tokens.includes(word)).length;
    const matchStrength = matchScore / brandWords.length;

    if (matchStrength >= 0.5 || line.toLowerCase().includes(medicine.brandName.toLowerCase())) {
      if (!matched.some(m => m.id === medicine.id)) matched.push(medicine);
    }
  }
}
```

4.5 Medicine Authentication & Anti-Counterfeit Verification Heuristics

Heuristic Scoring System: To protect users from counterfeit drugs, the platform includes a batch verification module. It runs batch numbers and manufacturer names through a scoring system mapping physical packaging criteria:

1. **Length Check:** Standard pharmaceutical batch numbers range from 5 to 15 characters. If the length is outside this boundary, risk score increases by \$+30\$.
2. **Character Set Check:** Batch numbers must be strictly alphanumeric with optional dashes. The presence of other special characters increases risk by \$+25\$.
3. **Numeric Purity:** Real manufacturer batch numbers incorporate letters to segment batches. A purely numeric string increases risk by \$+20\$.
4. **Suspect Sequence:** Checks for commonly faked or default patterns (e.g. '12345', 'ABCDE', 'FAKE', 'TEST'). If present, risk increases by \$+45\$.
5. **Manufacturer Verification:** Cross-references manufacturer against a verified Indian registry. Non-verification increases risk by \$+10\$.

The cumulative score is capped at 100. Results are classified as **Low Risk** (≤ 35), **Medium Risk** (36 - 65), or **High Risk** (> 65) with custom safety recommendations.

```
// Anti-Counterfeit Batch scoring in server.js
let riskScore = 15; // baseline risk
if (cleanBatch.length < 5 || cleanBatch.length > 15) riskScore += 30;
if (!/^[A-Z0-9-]+$/.test(cleanBatch)) riskScore += 25;
if (/^[0-9]+$/.test(cleanBatch)) riskScore += 20;
if (suspectSequences.some(seq => cleanBatch.includes(seq))) riskScore += 45;
if (manufacturer && !verifiedRegistry.includes(manufacturer.toLowerCase())) riskScore += 10;
riskScore = Math.min(100, riskScore);
```

4.6 Stock Depletion & Restock Forecaster

Logic: To predict when a pharmacy might run out of a generic drug, the system checks the medicine's availability status. If the stock status is 'Available', it checks the therapeutic category and current month. If the category is Respiratory or Antibiotics and the month corresponds to the Monsoon period (June to September), it sets a 'Medium Risk' level with a remaining stock estimate of \$12-22\$ days due to monsoon demand surges. Standard medicines default to \$45\$ remaining days. Simulated weekly sales velocity and estimated restock dates are dynamically calculated to model inventory.

5. Complete REST API Endpoints Specification

The backend server exposes the following endpoints to serve analytical widgets, coordinates queries, and predictions models.

Method & Endpoint	Input Params	Output JSON / Logic
GET /api/health	None	{ status: 'ok', time: 'ISO Timestamp' } Used for client latency checks.
GET /api/medicines	query (String, optional)	Returns matching list of brand/generic drugs. Evaluates against composition and EAN barcode.
GET /api/medicines/:id	id (Number in path)	Details of specific medicine, including side effects, warnings, dosage form, and safety schedules.
GET /api/analytics	None	Returns price summaries, list of top overpriced medicines, category average prices, and regional premiums.
GET /api/stores	lat, lng (Numbers) radius (Number, optional)	Finds pharmacies within radius using the Haversine formula . Returns sorted list with distance.
GET /api/trends	horizon (Number, default 6)	Provides historical volumes and forecasts generic demand using Linear Regression with seasonality .
POST /api/predict-price	Body: { brandPrice: Num, category: Str }	Returns generic price range and savings percent based on therapeutic markups and bulk adjustments.
POST /api/parse-prescription	Body: { text: String }	Parses raw OCR text. Matches brand drug tokens and returns net cost savings summaries.
GET /api/stock/predict/:id	id (Number in path)	Returns supply risk levels, remaining days of stock, weekly sales velocity, and simulated restock dates.
POST /api/fake-detection	Body: { batchNumber: Str, manufacturer: Str }	Runs the batch scoring heuristic system. Returns risk scores (0-100), risk levels, and guidelines.
POST /api/chat	Body: { message: Str, history: Arr }	Sends message to Gemini API. Falls back to local keyword responses if offline.
GET /api/disease-search	query (String)	Maps symptoms (e.g. fever, diabetes) to drug classes and returns top generic recommendations.

The React frontend implements fourteen service dashboards that communicate with the backend APIs. Below is a detailed, step-by-step description of how each feature functions.

1. Console Overview Dashboard:

A highly visual, animated landing dashboard (OverviewDashboard.tsx) serving as the main entry point. It displays a dynamic time-based greeting banner using assets ('/refill_tracker_header.png'), live counters for total pocket savings accumulated in the local Health Wallet database, a dynamic SVG radial progress savings dial that animates its stroke layout on mount, upcoming family medicine scheduler notifications, and responsive quick-launcher navigation cards for other key system modules.

2. Alternative Finder:

A central search dashboard (RecommendationSystem.tsx) allowing users to search by branded names (e.g. Crocin, Lipitor) or compositions. It triggers ``/api/medicines?query=...`` on input. It displays the generic chemical equivalents, pre-calculated savings (up to 80%), safety schedules, dosage forms, and side effects. It contains a **Voice Search** widget that uses the Web Speech API (speechRecognition) to capture audio medicine names, and a **Barcode Scanner** using device camera to parse EAN-13 barcodes, allowing users to scan packaging directly.

3. Prescription Cost Optimizer:

Allows users to upload images of physical prescriptions (PrescriptionOptimizer.tsx). The frontend runs Tesseract.js directly inside the browser, rendering a progress bar while extracting raw characters. The extracted text is then POSTed to ``/api/parse-prescription`` to isolate drug names and calculate potential savings. Users can download a compiled PDF saving report (using jsPDF) or share the savings summary via a generated QR code (QRPrescription component) which can be scanned by other devices.

4. Nearby Pharmacy Locator:

Locates nearby stores (NearbyShops.tsx). It uses the browser's Geolocation API (``navigator.geolocation.getCurrentPosition``) to fetch the user's GPS coordinates, sending them to ``/api/stores?lat=...&lng;=...``. The backend filters stores using the Haversine formula and returns them sorted by distance. It lists government Jan Aushadhi Kendras and private generic chemists, shows phone contacts, addresses, opening hours, and cross-references stock availability for the queried medicine, letting users verify stock before traveling.

5. Price Analysis Dashboard:

Displays financial analytics (PriceAnalysisDashboard.tsx) for market intelligence. It calls ``/api/analytics`` to render Recharts Bar and Line graphs showing top overpriced branded medicines, potential total savings margins, average pricing comparing branded vs generic across categories, and regional pricing differences in India.

6. Market Intelligence:

A forecasting dashboard (MarketIntelligence.tsx) for research. It queries ``/api/trends?horizon=6``. It draws an interactive Area Chart showing historical sales volumes alongside the Linear Regression forecasted demands. It also uses Recharts Pie charts to show market shares (e.g. Janaushadhi Kendra dominates at 42%, followed by private generic divisions).

7. Safety Analyzer:

A diagnostic tool (SafetyAnalyzer.tsx) focusing on chemical schedules. It details safety categories (OTC, Schedule H, Schedule H1), listing warnings and side effects. For Schedule H/H1 medicines, it displays a warnings banner urging the user to obtain a doctor's signature before purchasing.

8. Fake Medicine Detector:

An anti-counterfeiting verification widget (FakeDetector.tsx). The user enters a medicine's batch number and manufacturer. The client calls ``/api/fake-detection``, which runs the 5-rule heuristic checker. The screen displays the risk score with a colored gauge (Green for Low Risk, Orange for Medium Risk, Red for High Risk), outlines the specific warning factors found, and provides safety protocols.

9. Health Wallet:

A personal savings account tracker (HealthWallet.tsx). Users log their prescription purchases (medicine, quantity, price paid). The component saves this data to the browser's `localStorage`, displaying a cumulative wallet balance representing the total money saved by choosing generic alternatives. It displays progress cards and savings milestones.

10. Savings Calculator:

An interactive cost estimator (SavingsCalculator.tsx). Users use sliders to input their current monthly branded medicine costs. The tool calculates monthly, yearly, and 5-year savings, providing cost-saving visual metrics (e.g., equivalent to buying a mobile phone, paying school fees, or buying insurance).

11. Government Scheme Awareness:

An educational portal (SchemeAwareness.tsx) detailing national health schemes. It outlines PMBJP guidelines, eligibility parameters, step-by-step application checklists, and offers an interactive Ayushman Bharat (PM-JAY) checker.

12. GenMed AI Chatbot Assistant:

A floating chat widget (Chatbot.tsx) available across all pages. The user chats about side effects, alternatives, or schemes. The request POSTs to ``/api/chat``. If ``GEMINI_API_KEY`` is loaded in the backend ``.env`` file, the server queries the Gemini API for clinical, conversational responses. If offline or if the key is missing, the server runs a local keyword matching system to return pre-configured responses, ensuring the bot remains functional.

13. Generic Substitute Scanner:

An interactive barcode scanner simulation dashboard (GenericSubstituteScanner.tsx). Users can select from a pre-defined grid of popular Indian branded medicines to simulate scanning a physical package's barcode. The screen flashes on match success, and fetches full data including side-by-side cost comparisons, composition details, and stock statuses. It features an interactive monthly usage range slider that dynamically updates monthly and annual savings calculations based on the user's prescription consumption levels.

14. Medicine Reminder & Refill Tracker:

A calendar daily timeline and pill stock management dashboard (MedicineReminder.tsx). Users configure dosage reminders specifying medicine names, daily time slots (Morning, Afternoon, Evening, Night), and patient names for family schedules (Self, Father, Mother, Spouse). Dosage consumption can be checked off as 'taken' (which automatically decrements local inventory stock) or 'skipped'. If remaining pill counts fall below a custom threshold, the system displays dynamic low stock alerts and provides a reorder shortcut to find generic alternatives nearby.

7. Architectural Scalability & Recommendations

The current implementation of GenMed Hub™ demonstrates a highly functional local prototype. To transition this product to a national production scale, we recommend the following engineering roadmap:

1. Migration from Flat JSON to a Relational/NoSQL Database:

The mock JSON database system (``db.json``) is read and written synchronously via filesystem calls. While sufficient for single-user dev testing, this will fail under concurrent user loads. We recommend migrating to PostgreSQL (for ACID compliance on user transactions and purchase histories) or MongoDB (for flexible document storage of heterogeneous medicine compositions). Spatial queries for pharmacies can be optimized using PostgreSQL's PostGIS extension or MongoDB's 2dsphere indexing, replacing the javascript-level Haversine calculation.

2. Production OCR Pipeline Optimization:

Tesseract.js performs character extraction locally inside the client's browser. While this provides offline privacy, performance depends on user device hardware. A production application should support hybrid OCR: client-side processing for fast previews, with an option to offload complex images to a cloud OCR service (such as Google Cloud Vision API) for superior text extraction.

3. Enterprise AI Chat Integration:

The current chatbot relies on a basic fallback system if the Gemini key is absent. For production, we recommend deploying a Retrieval-Augmented Generation (RAG) pipeline. This would index official government PMBJP drug databases and CDSCO safety guidelines, embedding them in a vector database. The chat agent can then query this index to provide accurate, verified medical advice.

End of Technical Documentation.